LAB TEST-3

NAME: VARSHINI VEPURI

HALLTICKET NUMBER:2403A54106

#TASK-1:

Design and implement a solution using AI-assisted tools to address this challenge.

#Prompt:

Design an **AI-assisted backend API** for an **E-commerce platform** to manage products, carts, and orders . Integrate an **AI-powered recommendation system** to suggest similar products based on descriptions. Provide **Python source code**, **explanation of AI integration**, and **test results**
Use **in-memory data structures** and ensure responses are in **JSON format**. Demonstrate how AI improves the backend functionality.

#code:

```python
# ai_ecommerce.py
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Sample product data
products = [
    {"id": 1, "name": "Red Cotton T-Shirt", "desc": "comfortable red cotton tee"},
    {"id": 2, "name": "Blue Denim Jeans", "desc": "stylish blue denim jeans"},
    {"id": 3, "name": "Running Shoes", "desc": "lightweight running shoes for men"},
]

# --- AI Integration: TF-IDF based recommender ---
descriptions = [p["desc"] for p in products]
vectorizer = TfidfVectorizer()
vectors = vectorizer.fit_transform(descriptions)
similarity = cosine_similarity(vectors)

# --- Display recommendations ---
for i, p in enumerate(products):
    similar_idx = similarity[i].argsort()[-2]
    print(f"Product: {p['name']} → Recommended: {products[similar_idx]['name']}")
```

#output:

#Explanation:

The code uses **AI-assisted TF-IDF vectorization** to understand product descriptions.

Cosine similarity finds products with the most similar text features.

This forms the basis of an **AI-powered recommendation system** in an e-commerce backend

#Task-2:

Design and implement a solution using AI-assisted tools to address this challenge.

#Prompt:

Design and implement an **AI-assisted code refactoring solution** for a **Smart City application** that processes traffic, pollution, and energy data.
Use **AI tools** (like ChatGPT or Copilot) to enhance code readability, reduce redundancy, and improve modularity.
Provide **Python source code**, **explanation of AI integration**, and **test results/output screenshots**.
Show how AI helps optimize and modernize the existing code structure for better maintainability.

#code:

```python
class SmartCityAnalyzer:
    """AI-assisted refactoring: modular and extensible design"""

    def __init__(self, data):
        self.data = data

    def calculate_average(self):
        return sum(self.data) / len(self.data) if self.data else 0

    def analyze(self, data_type):
        handlers = {
            "traffic": self._analyze_traffic,
            "pollution": self._analyze_pollution,
            "energy": self._analyze_energy,
        }
        if data_type in handlers:
            return handlers[data_type]()
        else:
            return "Invalid data type!"

    def _analyze_traffic(self):
        avg = self.calculate_average()
        return f"Average traffic flow: {avg:.2f}"

    def _analyze_pollution(self):
        avg = self.calculate_average()
        return "⚠ High pollution alert!" if avg > 80 else f"Pollution level: {avg:.2f}"
```

```python
        return f"Total energy consumption: {total:.2f}"


# --- TESTING THE SOLUTION ---
if __name__ == "__main__":
    print("=== SMART CITY ANALYSIS RESULTS ===")
    traffic_data = [120, 100, 80, 90]
    pollution_data = [60, 85, 75]
    energy_data = [200, 300, 400]

    analyzer = SmartCityAnalyzer(traffic_data)
    print(analyzer.analyze("traffic"))

    analyzer = SmartCityAnalyzer(pollution_data)
    print(analyzer.analyze("pollution"))

    analyzer = SmartCityAnalyzer(energy_data)
    print(analyzer.analyze("energy"))

    print("\n--- AI Integration Explanation ---")
    print("This refactoring was guided by AI-assisted tools (like ChatGPT/Copilot),")
    print("which improved code readability, removed repetition, and made it modular.")
```

#output:

```
PROBLEMS 2  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    powershell

PS C:\Users\varshini\OneDrive\Desktop\AI LAB TEST-3> & C:\Users\varshini\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/varshini/OneDrive/Desktop/AI LAB TEST-3/Untitled-1.py"
=== SMART CITY ANALYSIS RESULTS ===
Average traffic flow: 97.50
Pollution level: 73.33
Total energy consumption: 900.00

--- AI Integration Explanation ---
This refactoring was guided by AI-assisted tools (like ChatGPT/Copilot),
which improved code readability, removed repetition, and made it modular.
PS C:\Users\varshini\OneDrive\Desktop\AI LAB TEST-3>
```

Do you want to install the recommended ' REditorSupport for the R language?

Install

#Explanation:

AI-assisted tools like **ChatGPT** and **Copilot** were used to refactor the old Smart City code for better structure and efficiency.
They helped identify repetitive logic and reorganized it into a **modular, class-based design**.
This improved **readability, reusability, and maintainability** of the code.
The refactored version is now easier to **extend and debug**, enhancing the system's overall performance.